MITHERA 프로젝트 세팅 진행 현황

서버·클라이언트 기본 세팅부터 PR 머지까지 **feat-iws → develop 머지 완료**

1 프로젝트 뼈대

`client`
`server`
`api`

npm workspaces

루트에서 클라이언트와 서버를 함께 관리
Node.js 24.x 지정

2 서버 구성

Express 5 + Mongoose

/api/health

헬스체크 엔드포인트
Jest·Supertest 테스트 2개 통과

3 DB 연결

MongoDB Atlas

.env MONGO_URI

Atlas 연결 확인 완료
연결 정보는 커밋 제외

4 클라이언트

Vite + React (JS)

/api 프록시

ESLint·Prettier 설정
개발 시 서버로 요청 전달

5 Vercel 서버리스

api/index.js 서버리스 진입점

DB 연결 재사용

요청마다 DB를 새로 열지 않도록
연결을 캐싱

6 배포 라우팅

vercel.json rewrites

/api/* → 서버

그 외 경로는 SPA(index.html)
CORS 설정 불필요

7 팀 협업 규칙

CLAUDE.md PR 리뷰 가이드

feat-이름 브랜치

develop 통합 → main 배포
이슈·PR 작성 규칙 정리

로컬 실행 확인

- ✓ 서버 5000번: /api/health → ok, DB connected
- ✓ 클라이언트 프록시로 같은 응답 확인
- ✓ 실행 명령: dev:server, dev:client (루트)

PR #5 리뷰 과정

- ✓ feat-iws → develop PR 생성
- ✓ 수정 요청 2회 반영 (템플릿 정리, icons.svg 삭제)
- ✓ 승인 후 머지 (389b362)

PR 1건

develop에 머지 완료

리뷰 수정 2회 · 원격 브랜치 정리 완료

보안: server/.env, .vercel 폴더는 커밋 제외 (.gitignore)

다음 단계: 팀원 브랜치 feat-guwon, feat-csw 작업 · main 릴리스 PR · Vercel 환경변수 등록

1 프로젝트 뼈대 (npm workspaces)

클라이언트와 서버를 저장소 하나에서 함께 관리합니다.

루트 package.json

```
{
  "name": "mithera",
  "private": true,
  "workspaces": ["client", "server"],
  "scripts": {
    "build": "npm run build -w client",
    "dev:client": "npm run dev -w client",
    "dev:server": "npm run dev -w server",
    "test": "npm run test -w server",
    "lint": "npm run lint --workspaces --if-present"
  },
  "engines": { "node": "24.x" }
}
```

명령어

```
# 저장소 루트(E:\vibecoding\40_mithera)에서 한 번만 실행
npm install

# Node.js 버전 확인 (24.x 이어야 함)
node -v
```

확인: 루트에 `node_modules`가 생기고, `client`와 `server` 양쪽의 패키지가 함께 설치되어 있으면 성공입니다.

2 서버 구성 (Express 5 + Mongoose)

헬스체크 API와 테스트·린트 환경을 만듭니다.

패키지 설치

```
# server 폴더에서 (처음 한 번)
cd server
npm init -y
npm install express mongoose dotenv cors
npm install -D jest supertest eslint prettier eslint-config-prettier @eslint/js globals

# 실행 스크립트 등록
npm pkg set scripts.dev="node --watch src/server.js" scripts.start="node src/server.js"
```

만든 파일과 역할

- `src/app.js`: Express 앱 설정, CORS, JSON 파싱, 404·에러 처리 (서버를 직접 띄우지 않음)
- `src/server.js`: DB 연결 후 `listen(5000)`으로 로컬 실행
- `src/routes/healthRoutes.js` → `src/controllers/healthController.js`: `GET /api/health`
- `tests/health.test.js`: 헬스체크와 404 테스트

테스트와 린트

```
cd server
npm test              # Tests: 2 passed 나와야 함
npm run lint          # 에러 없이 끝나야 함
```

확인: `npm test`에서 `Tests: 2 passed`가 나오면 성공입니다.

3 DB 연결 (MongoDB Atlas)

접속 정보는 `server/.env`에만 두고 Git에는 올리지 않습니다.

server/.env (커밋 금지)

```
PORT=5000
MONGO_URI=mongodb+srv://USER:PASSWORD@CLUSTER.mongodb.net/mithera?retryWrites=true&w=majority
```

예시 파일은 `server/.env.example`에 있습니다. 실제 값은 복사해서 채우세요.

Atlas에서 해야 할 설정

- **Database Access:** DB 사용자(아이디·비밀번호) 생성
- **Network Access:** 접속할 IP 등록. 배포까지 고려하면 `0.0.0.0/0` 허용이 필요합니다 (보안 트레이드오프 있음)
- **DB 이름:** URI 끝에 `/mithera`를 넣어야 `mithera` DB에 연결됩니다. 빠지면 `test`로 연결됩니다.

연결 확인 명령어

```
cd server
npm run dev:server    # 루트에서 실행해도 됨

# 정상 로그 예시
MongoDB connected: ...mongodb.net/mithera
Server listening on http://localhost:5000
```

확인: `MongoDB connected`에 DB 이름이 `mithera`로 보이면 성공입니다.

4 클라이언트 (Vite + React, JavaScript)

화면을 만들고, `/api` 요청을 서버로 넘기는 개발 프록시를 설정합니다.

생성과 패키지 설치

```
# client 폴더(비어 있는 상태)에서 실행
cd client
npm create vite@latest . -- --template react --no-interactive
npm install
npm install -D eslint @eslint/js globals eslint-plugin-react-hooks eslint-plugin-react-refresh eslint-config-prettier prettier
npm pkg set scripts.lint="eslint ." scripts.format="prettier --write ."
```

vite.config.js 의 프록시 설정

```
export default defineConfig({
  plugins: [react()],
  server: {
    proxy: {
      '/api': 'http://localhost:5000',
    },
  },
})
```

개발 중에는 `/api` 요청이 서버(5000)로 전달됩니다. 서버 포트를 바꾸면 이 주소도 함께 바꿔야 합니다.

정리한 것

- 템플릿 기본 린터(oxlint)와 `@types/*`는 제거하고 ESLint + Prettier로 교체
- 모든 세팅 후 `npm run lint`, `npm run build` 실행

확인: `npm run build`가 `✓ built`로 끝나고, `npm run lint`에 에러가 없으면 성공입니다.

5 Vercel 서버리스 진입점

Vercel은 `listen()`을 호출하지 않으므로, 함수 형태로 Express 앱을 내보내야 합니다.

api/index.js (루트)

```
const app = require('../server/src/app');
const connectDB = require('../server/src/config/db');

module.exports = async (req, res) => {
  await connectDB();
  return app(req, res);
};
```

server/src/config/db.js: 연결 재사용

- 이미 연결된 상태(`readyState === 1`)면 바로 반환
- 연결 중인 Promise를 변수에 보관해서, 같은 함수 인스턴스 안에서 연결을 한 번만 만듦
- 연결이 실패하면 보관값을 비워서 다음 요청에서 다시 시도

로컬에서 로드 확인

```
node -e "require('./api/index.js'); console.log('api entry loads')"
```

이 명령은 파일이 문법 오류 없이 불러와지는지만 확인합니다. 실제 Vercel 동작은 배포 후에 확인합니다.

주의: 루트 `api/index.js`는 `server/`의 코드를 불러오므로, 루트 `package.json`의 workspaces로 의존성이 함께 설치되어야 합니다.

6 배포 라우팅 (vercel.json)

같은 도메인에서 `/api`는 서버로, 나머지는 클라이언트 화면으로 보냅니다.

vercel.json (루트)

```
{
  "buildCommand": "npm run build",
  "outputDirectory": "client/dist",
  "rewrites": [
    { "source": "/api/(.*)", "destination": "/api" },
    { "source": "/(.*)", "destination": "/index.html" }
  ]
}
```

클라이언트에서 호출하는 방법

```
fetch('/api/health')   # 상대 경로만 쓰면 됨 (도메인·CORS 설정 불필요)
```

Vercel 대시보드에서 할 일 (배포 전)

- **Import:** GitHub 저장소를 Vercel에 연결, Root Directory는 저장소 루트
- **Environment Variables:** `MONGO_URI` 등록 (`.env`는 배포에 포함되지 않음)
- **Production 브랜치:** `main`. `main`에 머지되면 자동 배포

배포 후 확인: `https://도메인/api/health` 가 `{"status":"ok","db":"connected"}`을 반환하면 성공입니다.

7 팀 협업 규칙과 브랜치 흐름

CLAUDE.md와 docs/pr-review-guide.md에 규칙을 정리했습니다.

브랜치 흐름

```
feat-iws    ┐
feat-guwon ─┼─PR─▶ develop ─PR─▶ main (Vercel 배포)
feat-csw    ┘

main, develop 에는 직접 push 하지 않습니다.
```

작업 시작 명령어 (매번)

```
git switch develop
git fetch origin
git switch feat-이름              # 본인 브랜치 (예: feat-iws)
git merge origin/develop          # develop 최신 내용 반영
```

작업 후 PR 올리기

```
git status                        # 올라갈 파일 확인 (.env 가 없어야 함)
git add 파일명
git commit -m "작업 내용 요약"
git push origin feat-이름
```

GitHub에서 base는 `develop`, compare는 `feat-이름`으로 PR을 엽니다. 본문에 `Closes #번호`를 적으면 머지 시 이슈가 닫힙니다.

확인: `git status -sb`에서 `## feat-이름...origin/feat-이름`처럼 원격과 맞춰져 있으면 준비 완료입니다.

로컬 실행 확인 (서버 + 클라이언트)

터미널 두 개로 서버와 클라이언트를 띄우고, 응답을 확인합니다.

1. 사전 확인

```
node -v          # v24.x
npm -v
git status       # 필요 없는 변경이 없는지
```

- `server/.env`에 `MONGO_URI`가 있는지 확인 (값은 출력하지 않기)
- 포트 5000, 5173이 비어 있는지 확인 (아래 문제 해결 참고)

2. 이미 떠 있는 프로세스 확인

```
netstat -ano | findstr :5000
netstat -ano | findstr :5173

# 필요 없는 프로세스 종료 (PID는 위 결과에서 확인)
taskkill /PID PID번호 /F
```

3. 터미널 1: 서버 실행 (루트에서)

```
npm run dev:server

# 기대 로그
◇ injected env (2) from .env
MongoDB connected: ...mongodb.net/mithera
Server listening on http://localhost:5000
```

4. 터미널 2: 클라이언트 실행 (루트에서)

```
npm run dev:client

# 기대 로그 (포트가 5173이 아니면 표시된 주소 사용)
Local:   http://localhost:5173/
```

로컬 실행 확인 (응답 검증과 문제 해결)

서버 직접 호출, 클라이언트 프록시 호출, 테스트까지 확인합니다.

5. 응답 확인 (새 터미널에서)

```
# 서버 직접 호출
curl http://localhost:5000/api/health
# 기대: {"status":"ok","db":"connected"}

# 클라이언트 프록시 경유 (5173 또는 표시된 포트)
curl http://localhost:5173/api/health
# 기대: 위와 같은 응답

# 없는 경로 (404 처리 확인)
curl http://localhost:5000/api/nope
# 기대: {"message":"Not Found"}
```

Windows PowerShell에서는 `curl`이 다른 명령으로 연결될 수 있으니 `curl.exe`를 쓰거나, 브라우저 주소창에 같은 주소를 입력해도 됩니다.

6. 테스트, 린트, 빌드 (선택)

```
npm test          # 루트에서: Tests: 2 passed
npm run lint      # 에러 없이 끝남
npm run build     # 클라이언트 빌드: ✓ built
```

7. 문제 해결

- **EADDRINUSE 오류:** 포트를 쓰는 프로세스가 있음 → `netstat -ano | findstr :5000`로 PID 확인 후 `taskkill /PID PID번호 /F`
- **MONGO_URI is not set:** `server/.env` 파일과 변수 이름 확인
- **DB 연결 실패:** Atlas Network Access에 현재 IP가 있는지, 사용자 비밀번호가 맞는지 확인
- **클라이언트에서 /api 실패:** 서버 터미널이 먼저 떠 있는지 확인. `PORT`를 5000이 아닌 값으로 바꾸면 프록시가 연결되지 않음

종료: 각 터미널에서 `Ctrl + C`. 백그라운드로 띄운 경우에는 위의 `netstat` + `taskkill`로 정리합니다.

R PR 리뷰 과정 (로컬 검증 후 GitHub 댓글·승인)

예시: PR #5 (feat-iws → develop). 번호와 브랜치 이름은 실제 PR에 맞게 바꿉니다.

1~2. PR 번호 확인, 작업 정리

```
gh pr list                      # 열린 PR 목록과 번호 확인
gh auth status                  # 리뷰하는 계정이 작성자와 다른지 확인

git status                      # 커밋 안 된 변경이 있으면
git stash push -m "리뷰 전 임시저장"   # 잠시 치워 두기 (나중에 pop)
```

3. PR 브랜치를 로컬로 가져오기

```
# 방법 A: GitHub CLI (리뷰용 이름을 따로 지정해서 본인 브랜치와 충돌 방지)
gh pr checkout 5 --branch review/pr-5

# 방법 B: GitHub CLI 없이
git fetch origin pull/5/head:pr-5
git switch pr-5
```

방법 A에서 `--branch`를 빼면 PR의 원래 브랜치 이름(예: feat-iws)으로 로컬 브랜치가 생겨서, 본인 브랜치와 겹칠 수 있습니다.

4. 변경 범위 파악

```
git log --oneline develop..HEAD
git diff --stat develop...HEAD
```

5. 직접 실행해 보기

```
npm install
npm test
npm run lint
npm run dev:server     # 다른 터미널에서 curl로 health 확인
```

6. AI 리뷰 (Claude Code)

가져온 브랜치에서 Claude Code를 열고 입력합니다. 결과는 터미널에만 보이고 GitHub에는 올라가지 않습니다.

```
/code-review
```

R PR 리뷰 과정 (댓글·승인·머지·복귀)

GitHub에 리뷰 결과를 남기고, 원래 작업 브랜치로 돌아갑니다.

7. GitHub에 댓글 남기기

웹 (권장): PR 페이지 → `Files changed` → 줄 번호 옆 `+`로 줄 댓글, 전체 의견은 `Conversation` 입력창

```
# 터미널에서 (GitHub CLI)
gh pr comment 5 --body "전체 의견 내용"
```

8. 승인 또는 수정 요청

웹: `Files changed` → 오른쪽 위 `Submit review` → `Approve` / `Request changes` 선택 → `Submit`

```
# 터미널에서
gh pr review 5 --approve
gh pr review 5 --request-changes --body "수정이 필요한 이유"
```

9. 머지 (리더 또는 지정된 담당자)

```
# 웹: PR 페이지의 Merge pull request 버튼
# 터미널: 머지 커밋 방식 (헤드 커밋을 고정해서 승인 이후 변경 방지)
gh pr merge 5 --merge --match-head-commit 전체해시
```

전체 해시는 `git rev-parse feat-iws`로 확인합니다. 승인이 없거나 승인 후 새 커밋이 올라오면 머지가 막힙니다.

10. 원래 브랜치로 돌아가기

```
git switch feat-guwon              # 본인 작업 브랜치로 복귀
git branch -D review/pr-5          # 임시 리뷰 브랜치 삭제 (방법 B면 pr-5)
git stash pop                      # 3단계에서 stash 했다면 복구
git status -sb
```

주의: `git branch -D`는 되돌릴 수 없습니다. 리뷰 브랜치 이름이 맞는지 `git branch`로 먼저 확인하세요.